Project Progress Report

Evan & Zhang
1009896141, evanz.zhang@mail.utoronto.ca
GitHub: APS360-Project

Project Description

The goal of this project is to develop a multitask deep learning system to identify vehicle license plate, make, model, and year, providing secondary verification layer to standard Automated License Plate Recognition (ALPR) systems. This multi-attribute classification is useful for automating parking, toll management, and security enforcement. By using Convolutional Neural Networks (CNNs) within a multitask learning framework, the system can efficiently extract alphanumeric details and geometric shapes of vehicles in varying weather and lighting conditions. The model's flowchart is shown in Figure 1.

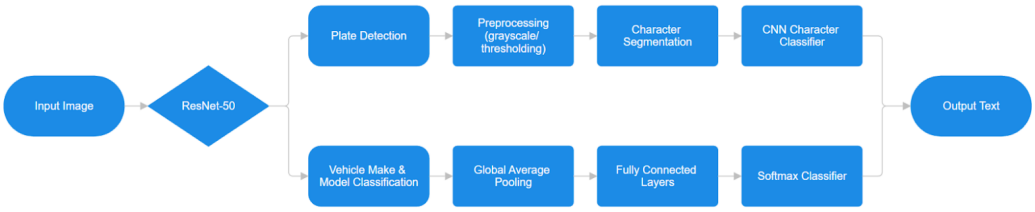

Figure 1: System architecture and workflow diagram.

The input to the deep learning model is a cropped, 224x224 pixel color image of a single vehicle. The model then outputs the prediction on the vehicle's exact make, model, and year. Deep learning is suitable because CNNs, like Res-Net50, are purpose built to recognize complex visual patterns. This allows it to adapt to real-world variations, where traditional machine learning methods struggle to manage the extra noise.

The project in its current state is focused on vehicle make and model recognition (VMMR).

Data Processing

The primary dataset for the VMMR portion of this project is the Stanford Cars dataset, which contains 8,144 training images distributed across 196 specific vehicle classes. The original dataset provided the image annotations, including class labels and specific bounding box coordinates within the MATLAB arrays.

Because our pipeline is built in Python, the first step was extract the information within the .MAT files to a human-readable standardized CSV file (train_annotations.csv). This CSV acts as a guests list for the model, allowing us to link each .jpg file to their metadata during training.

To train the model and evaluate without data leakage, the dataset is divided in an 80% training (6,515 images) and 20% validation split. Rather than physically moving images into separate folders, I used scikit-learn to split the master CSV file into two separte CSV lists (train_split.csv and val_split.csv). While the split is done at random, I applied a fixed random seed (24) during this split to make the results reproducible.

Data augmentation is handled dynamically. Instead of pre-processing and saving thousands of images on the hard drive, the DataLoader applies a on-the-fly transformation to each batch of images before they are processed. The transformation consists of the following steps:

1. Cropping: Using the bounding box coordinates from the CSV, the Pillow library crops the original image to bound the vehicle.
2. Standardization: All cropped and augmented images are resized to a uniform 224x224 pixel square and normalized using standard ImageNet mean and standard deviation values. The validation set undergoes the resizing and normalization, but bypasses the random augmentation to ensure the evaluation metric remains consistent.
3. Dynamic Augmentation (training set only): introduce controlled randomness to prevent the model from overfitting. A 50% probability of a horizontal flip is applied, as well as a randomized color jitter (adjust brightness and contrast +- 20%). This effectively multiplies our dataset, ensuring the model does not sees the exact same car twice.

An example of this is shown in Figure 2

Figure 2: Data preprocessing pipeline and augmentation example using an Audi S5.

For the final testing of this project, I intend to evaluate the model on unseen, real-world data. The primary plan is to photograph parked vehicles around Toronto, capturing standard street parking scenarios. If I cannot source a sufficient volume of images, the dataset will be supplemented by the unlabeled cars_test dataset provided in the original Stanford archive.

Baseline Model

For the baseline model, I implemented a traditional Histogram of Oriented Gradients (HOG) feature extractor paired with a Linear Support Vector Machine (SVM) classifier. This works for vehicle identification as a car's make and model is largely defined by its geometric shapes. While deep learning process the pixels directly, this approach requires feature extraction, achieved by HOG. HOG isolates the physical edges while ignoring irrelevant information like paint color, creating a "wireframe" of the vehicle. The SVM then uses these wireframes to calculate rigid boundaries between the 196 classes.

To ensure a fair comparison with the primary neural network, the baseline model was trained and evaluated on the same 80/20 data split (train_split.csv and val_split.csv). However, the images are transformed differently. All images were converted to grayscale, and resized to a uniform 128x64 pixels.

The SVM was trained on the extracted HOG features. The results highlight the limitations of traditional machine learning on finer visual classification tasks:

- Training accuracy: 99.89%
- Validation accuracy: 14.79%

With 196 vehicle classes, random guessing would yield an accuracy of 0.51%. The baseline's 14.79% validation accuracy indicate that extracting shape and edge data does help a model differentiate vehicles well above random guessing. However, the massive gap between training and validation indicate severe overfitting. The SVM likely memorized the exact gradients but failed to generalize to unseen images. The HOG + SVM model proves that it is able to distinguish between very different

vehicle geometry, for example, between a pick-up truck and a coupe. However, it fails on intra-class distinctions, where due to the scaled down input the model loses subtle distinctions. The aggressive downscaling to 128x64 was done to reduce computation load.

Primary Model

For the primary model, I implemented a modified ResNet-50 architecture using PyTorch. ResNet-50 is highly appropriate for VMMR as its residual connections allow for deep feature extraction without suffering the vanishing gradient problem. Transfer learning was used to optimize training time and computational resources. The ResNet-50 backbone was pre-trained on the ImageNet dataset, and I modified the final fully connected (fc) classification, replacing the default ImageNet output with a linear layer specified to our 196 vehicle classes.

- Loss Function: Cross-Entropy Loss
- Optimizer: Adam, learning rate (lr) = 0.001
- Batch size: 32

I ran an initial 5 epoch training test to evaluate the model's baseline feasibility. The results prove the viability of this approach.

- Initial training loss (Epoch 1): 5.1192
- Final training loss (Epoch 5): 0.5228
- Final validation accuracy: 78.88%

The model's learning curve is shown Figure 3

Figure 3: ResNet-50 Training Loss and Validation Accuracy over 5 epochs.

Unlike the HOG + SVM approach which stalled at 14.79% validation accuracy, the ResNet-50 model was able to successfully generalize its learning to unseen data. The progressive decrease in training loss and increase in validation accuracy shows that the model is not simply memorizing the training set. The 78.88% accuracy achieved in only 5 epochs far exceed random guessing (0.51%), and strongly outperforms the baseline model. As the training loss continued to decrease by the end of the 5th epoch, further training can achieve even higher accuracy.

The current 78.88% accuracy validates the feasibility of the ResNet-50 approach, and will be extended to 20 epochs and gathering Toronto-based street data for final testing. The ALPR part of the project will follow a similar trajectory.

References

[1] Krause, J., Stark, M., Deng, J., and Fei-Fei, L. (2013). 3D Object Representations for Fine-Grained Categorization. In *4th IEEE Workshop on 3D Representation and Recognition (3dRR-13)*.

[2] He, K., Zhang, X., Ren, S., and Sun, J. (2016). Deep Residual Learning for Image Recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770-778.

[3] Dalal, N. and Triggs, B. (2005). Histograms of Oriented Gradients for Human Detection. In *2005 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'05)*, Vol. 1, pp. 886-893.